Test Driven Development

More JUnit Tests for the Shop app

Produced by: Dr. Siobhán Drohan
Mairead Meagher
Siobhan Roche

Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

Department of Computing and Mathematics
http://www.setu.ie/

Topic List

- Product and ProductTest.java

- JUnit Testing of Store.java

- Testing Driver.java

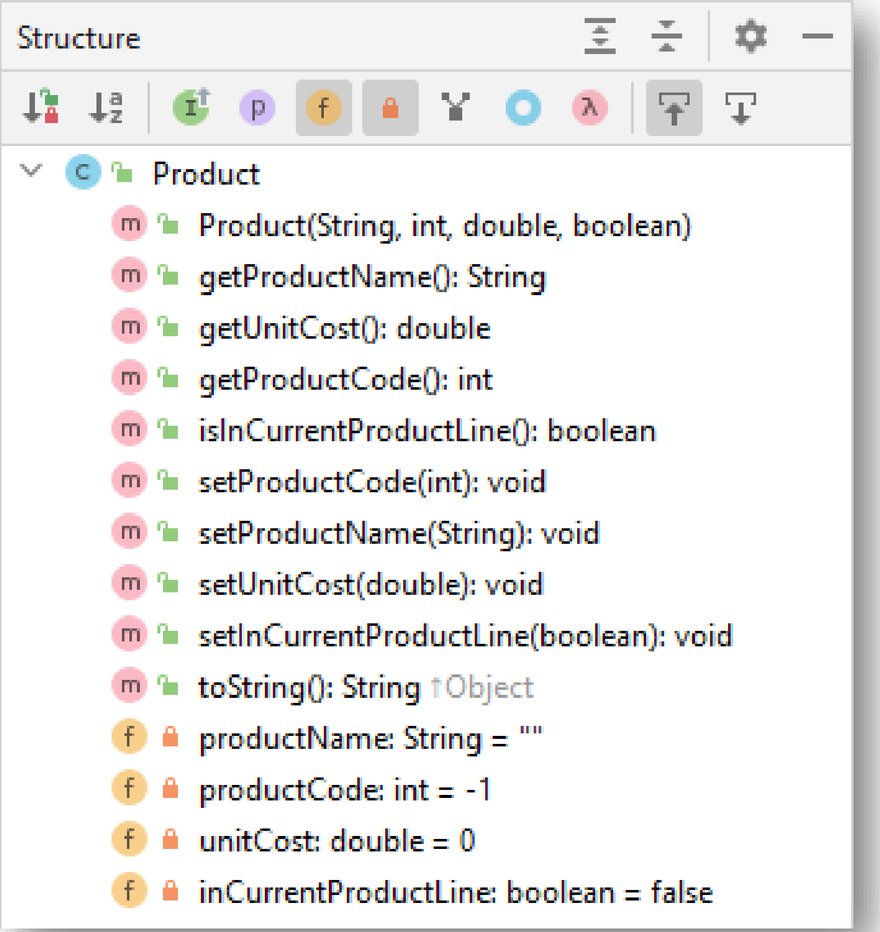
Product.java

```java
package models;

import utils.Utilities;

public class Product {

   private String productName = "";
   private int productCode = -1;
   private double unitCost = 0;
   private boolean inCurrentProductLine = false;

   public Product(String productName, int productCode, double unitCost, boolean inCurrentPro
      this.productName = Utilities.truncateString(productName, 20);
      setProductCode(productCode);
      this.unitCost = unitCost;
      this.inCurrentProductLine = inCurrentProductLine;
   }

  public String toString()
   {
      return "Product description: " + productName
            + ", product code: " + productCode
            + ", unit cost: " + unitCost
            + ", currently in product line: " + Utilities.booleanToYN(inCurrentProductLine);
   }

}
```

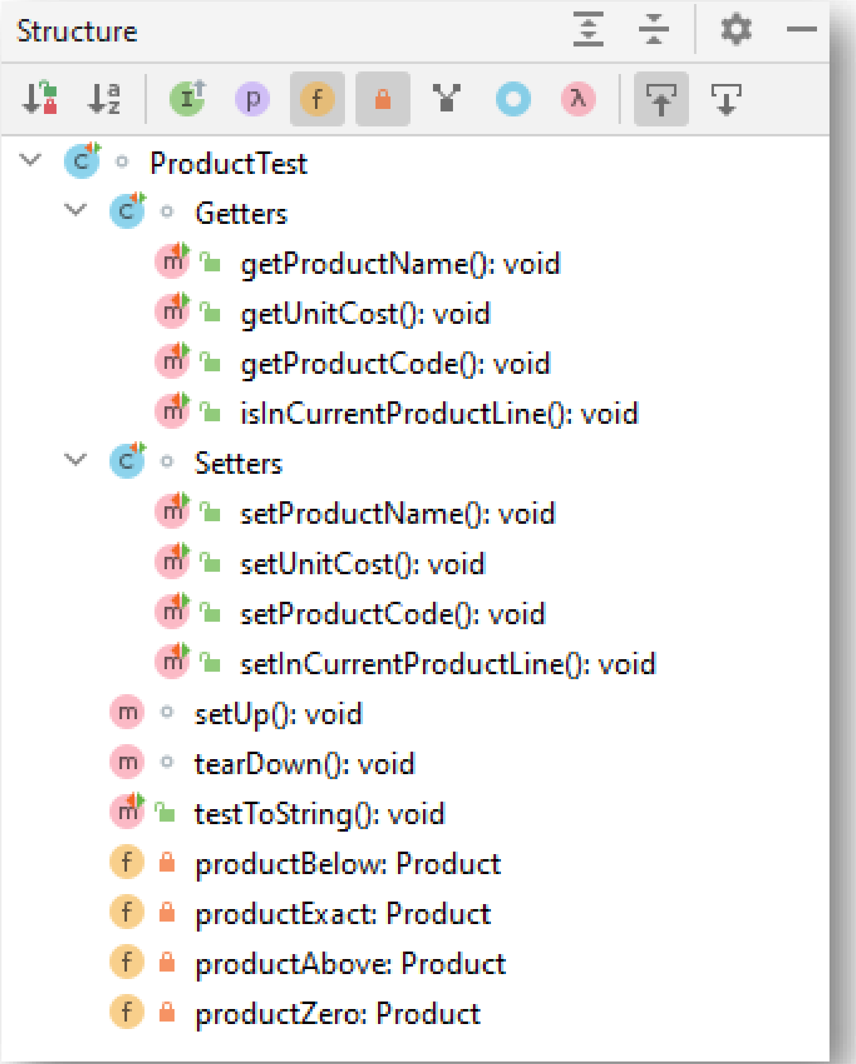
```java
public void setProductCode(int productCode) {
   if (Utilities.validRange(productCode, 1000, 9999)) {
      this.productCode = productCode;
   }
}

public void setProductName(String productName) {
   if (Utilities.validateStringLength(productName, 20)) {
      this.productName = productName;
   }
}

public void setUnitCost(double unitCost) {
   this.unitCost = unitCost;
}

public void setInCurrentProductLine(boolean inCurrentProductLine)
   this.inCurrentProductLine = inCurrentProductLine;
}
```

```java
 public String getProductName(){
    return productName;
 }
public double getUnitCost(){
    return unitCost;
 }
 public int getProductCode() {
    return productCode;
 }
 public boolean isInCurrentProductLine() {
    return inCurrentProductLine;
 }
```

Product.java



ProductTest.java



ProductTest.java

ProductTest.java × Product.java ×

```java
package models;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class ProductTest {

    private Product productBelow, productExact, productAbove, productZero;

    @BeforeEach
    void setUp() {...}

    @AfterEach
    void tearDown() { productBelow = productExact = productAbove = productZero = null; }

    @Nested
    class Getters {...}

    @Nested
    class Setters {...}

    @Test
    void testToString() {...}
}
```

Structure

- ProductTest
  - Getters
    - getProductName(): void
    - getUnitCost(): void
    - getProductCode(): void
    - isInCurrentProductLine(): void
  - Setters
    - setProductName(): void
    - setUnitCost(): void
    - setProductCode(): void
    - setInCurrentProductLine(): void
  - setUp(): void
  - tearDown(): void
  - testToString(): void
  - productBelow: Product
  - productExact: Product
  - productAbove: Product
  - productZero: Product

ProductTest.java

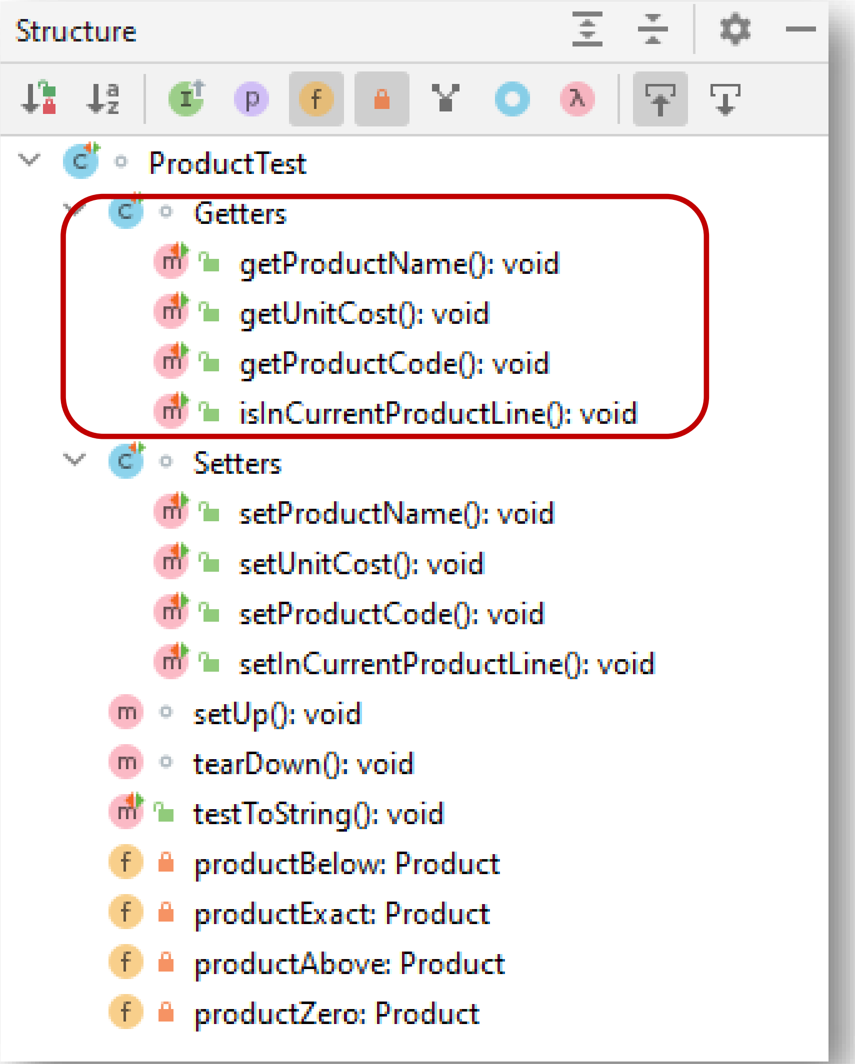
```java
package models;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class ProductTest {

    private Product productBelow, productExact, productAbove, productZero;

    @BeforeEach
    void setUp() {
        //name, 19 chars, code 999, unitCost 1, inCurrentProductLine true.
        productBelow = new Product( productName: "Television 42Inches",  productCode: 999,  unitCost: 1,  inCurrentProductLine: true);
        //name, 20 chars, code 1000, unitCost 999, inCurrentProductLine true.
        productExact = new Product( productName: "Television 50 Inches",  productCode: 1000,  unitCost: 999,  inCurrentProductLine: true);
        //name, 21 chars, code 10000, unitCost 1000, inCurrentProductLine false.
        productAbove = new Product( productName: "Television 60 Inches.",  productCode: 10000,  unitCost: 1000,  inCurrentProductLine: false);
        //name, 0 chars, code 9999, unitCost 0, inCurrentProductLine false.
        productZero = new Product( productName: "",  productCode: 9999,  unitCost: 0,  inCurrentProductLine: false);
    }

    @AfterEach
    void tearDown() {
        productBelow = productExact = productAbove = productZero = null;
    }
```

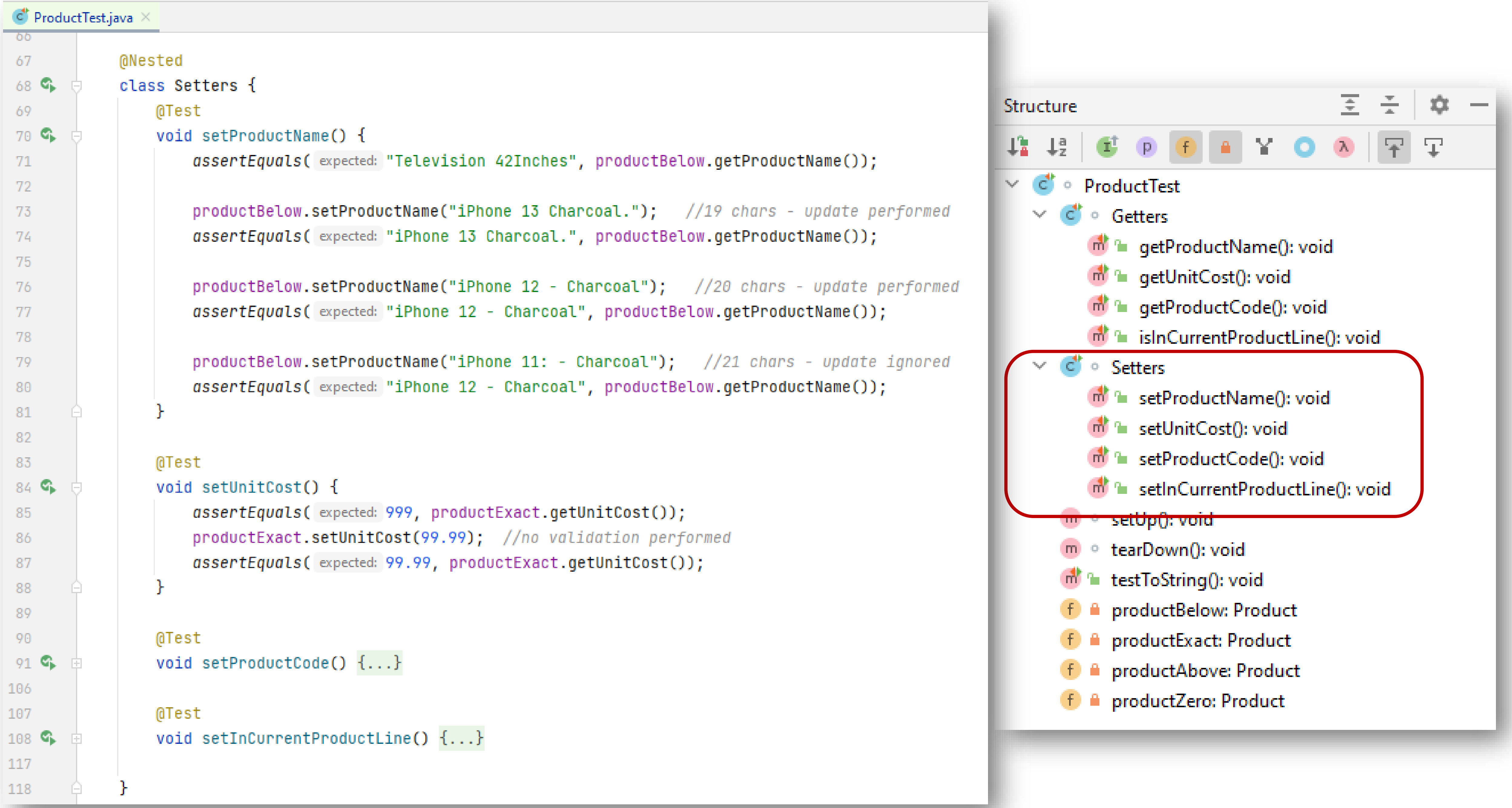
Structure

- ProductTest
  - Getters
    - getProductName(): void
    - getUnitCost(): void
    - getProductCode(): void
    - isInCurrentProductLine(): void
  - Setters
    - setProductName(): void
    - setUnitCost(): void
    - setProductCode(): void
    - setInCurrentProductLine(): void
  - setUp(): void
  - tearDown(): void
  - testToString(): void
  - productBelow: Product
  - productExact: Product
  - productAbove: Product
  - productZero: Product

ProductTest.java

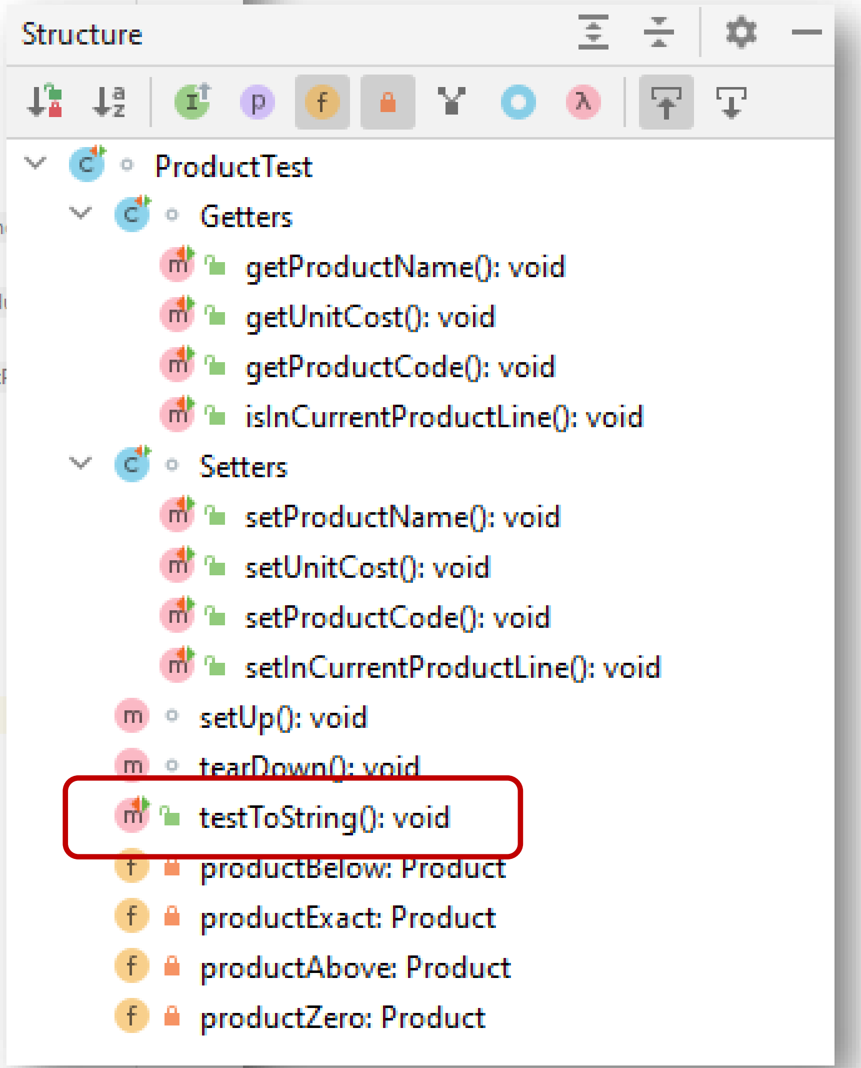
```java

    @Nested
    class Getters {

        @Test
        void getProductName() {
            assertEquals( expected: "Television 42Inches", productBelow.getProductName());
            assertEquals( expected: "Television 50 Inches", productExact.getProductName());
            assertEquals( expected: "Television 60 Inches", productAbove.getProductName());
            assertEquals( expected: "", productZero.getProductName());
        }

        @Test
        void getUnitCost() {
            assertEquals( expected: 1, productBelow.getUnitCost());
            assertEquals( expected: 999, productExact.getUnitCost());
            assertEquals( expected: 1000, productAbove.getUnitCost());
            assertEquals( expected: 0, productZero.getUnitCost());
        }

        @Test
        void getProductCode() {
            assertEquals( expected: -1, productBelow.getProductCode());
            assertEquals( expected: 1000, productExact.getProductCode());
            assertEquals( expected: -1, productAbove.getProductCode());
            assertEquals( expected: 9999, productZero.getProductCode());
        }

        @Test
        void isInCurrentProductLine() {
            assertTrue(productBelow.isInCurrentProductLine());
            assertTrue(productExact.isInCurrentProductLine());
            assertFalse(productAbove.isInCurrentProductLine());
            assertFalse(productZero.isInCurrentProductLine());
        }
    }
```




ProductTest.java
@Nested
class Setters {
@Test
void setProductName() {
assertEquals(expected: "Television 42Inches", productBelow.getProductName());
productBelow.setProductName("iPhone 13 Charcoal."); //19 chars - update performed
assertEquals(expected: "iPhone 13 Charcoal.", productBelow.getProductName());
productBelow.setProductName("iPhone 12 - Charcoal"); //20 chars - update performed
assertEquals(expected: "iPhone 12 - Charcoal", productBelow.getProductName());
productBelow.setProductName("iPhone 11: - Charcoal"); //21 chars - update ignored
assertEquals(expected: "iPhone 12 - Charcoal", productBelow.getProductName());
}
@Test
void setUnitCost() {
assertEquals(expected: 999, productExact.getUnitCost());
productExact.setUnitCost(99.99); //no validation performed
assertEquals(expected: 99.99, productExact.getUnitCost());
}
@Test
void setProductCode() {...}
@Test
void setInCurrentProductLine() {...}
}
Structure
ProductTest
Getters
getProductName(): void
getUnitCost(): void
getProductCode(): void
isInCurrentProductLine(): void
Setters
setProductName(): void
setUnitCost(): void
setProductCode(): void
setInCurrentProductLine(): void
setUp(): void
tearDown(): void
testToString(): void
productBelow: Product
productExact: Product
productAbove: Product
productZero: Product

ProductTest.java

```
import static org.junit.jupiter.api.Assertions.*;

class ProductTest {

    private Product productBelow, productExact, productAbove, productZero;

    @BeforeEach
    void setUp() {
        //name, 19 chars, code 999, unitCost 1, inCurrentProductLine true.
        productBelow = new Product( productName: "Television 42Inches",  productCode: 999,  unitCost: 1,  inCurrentProductLin
        //name, 20 chars, code 1000, unitCost 999, inCurrentProductLine true.
        productExact = new Product( productName: "Television 50 Inches",  productCode: 1000,  unitCost: 999,  inCurrentProd
        //name, 21 chars, code 10000, unitCost 1000, inCurrentProductLine false.
        productAbove = new Product( productName: "Television 60 Inches.",  productCode: 10000,  unitCost: 1000,  inCurrent
        //name, 0 chars, code 9999, unitCost 0, inCurrentProductLine false.
        productZero = new Product( productName: "",  productCode: 9999,  unitCost: 0,  inCurrentProductLine: false);
    }

    @AfterEach
    void tearDown() { productBelow = productExact = productAbove = productZero = null; }

    @Nested
    class Getters {...}

    @Nested
    class Setters {...}

    @Test
    void testToString() {
        String toStringContents = productExact.toString();
        assertTrue(toStringContents.contains("Product description: " + productExact.getProductName()));
        assertTrue(toStringContents.contains("product code: " + productExact.getProductCode()));
        assertTrue(toStringContents.contains("unit cost: " + productExact.getUnitCost()));
        assertTrue(toStringContents.contains(" currently in product line: Y"));
    }
}
```



Topic List

- Product and ProductTest.java
- JUnit Testing of Store.java
- Testing Driver.java

Store.java

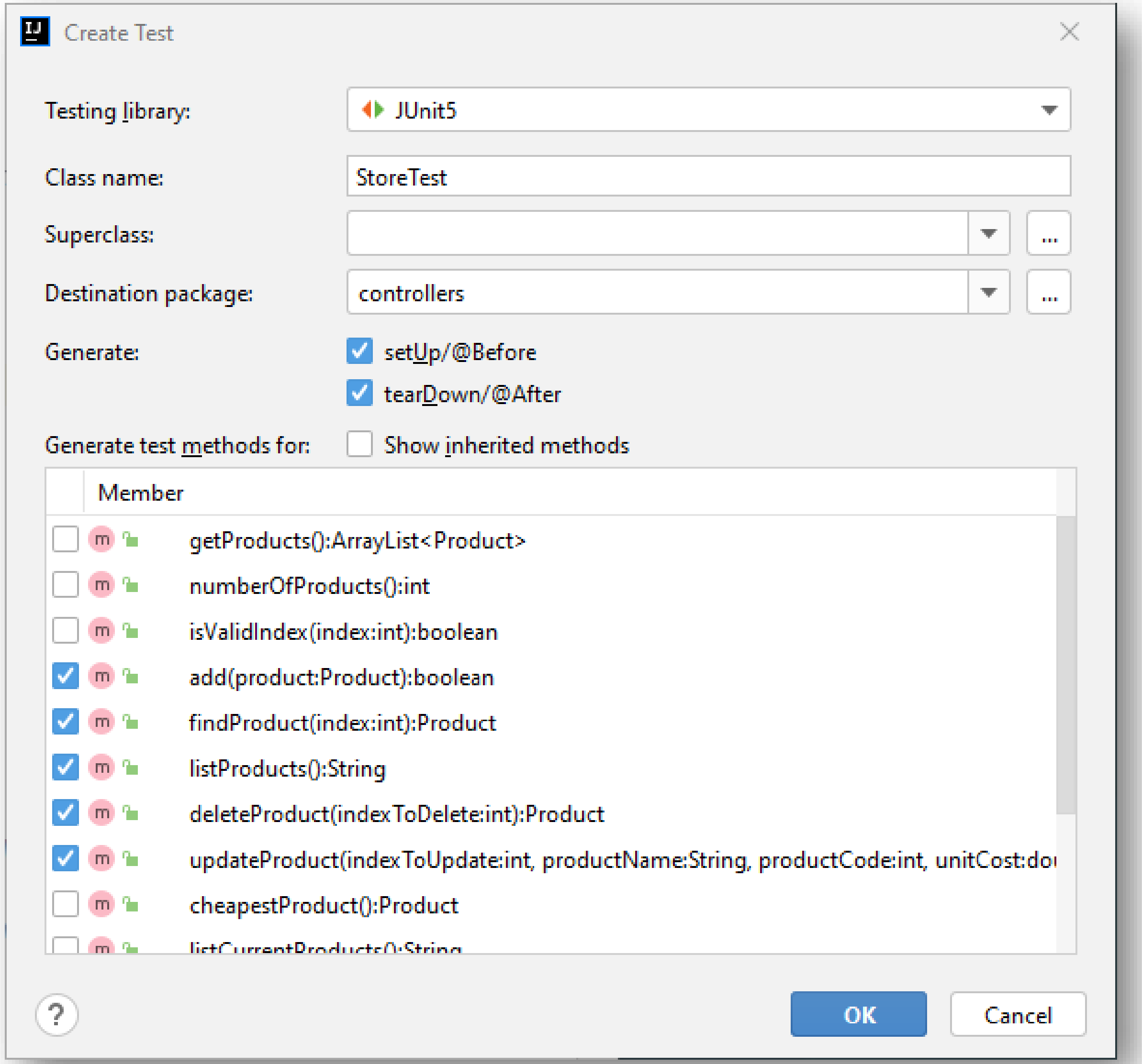
- Store
  - Store()
  - getProducts(): ArrayList<Product>
  - numberOfProducts(): int
  - isValidIndex(int): boolean
  - add(Product): boolean
  - findProduct(int): Product
  - listProducts(): String
  - deleteProduct(int): Product
  - updateProduct(int, String, int, double, boolean): boolean
  - cheapestProduct(): Product
  - listCurrentProducts(): String
  - averageProductPrice(): double
  - listProductsAboveAPrice(double): String
  - load(): void
  - save(): void
  - products: ArrayList<Product>

We will look at writing tests for a few of these methods.

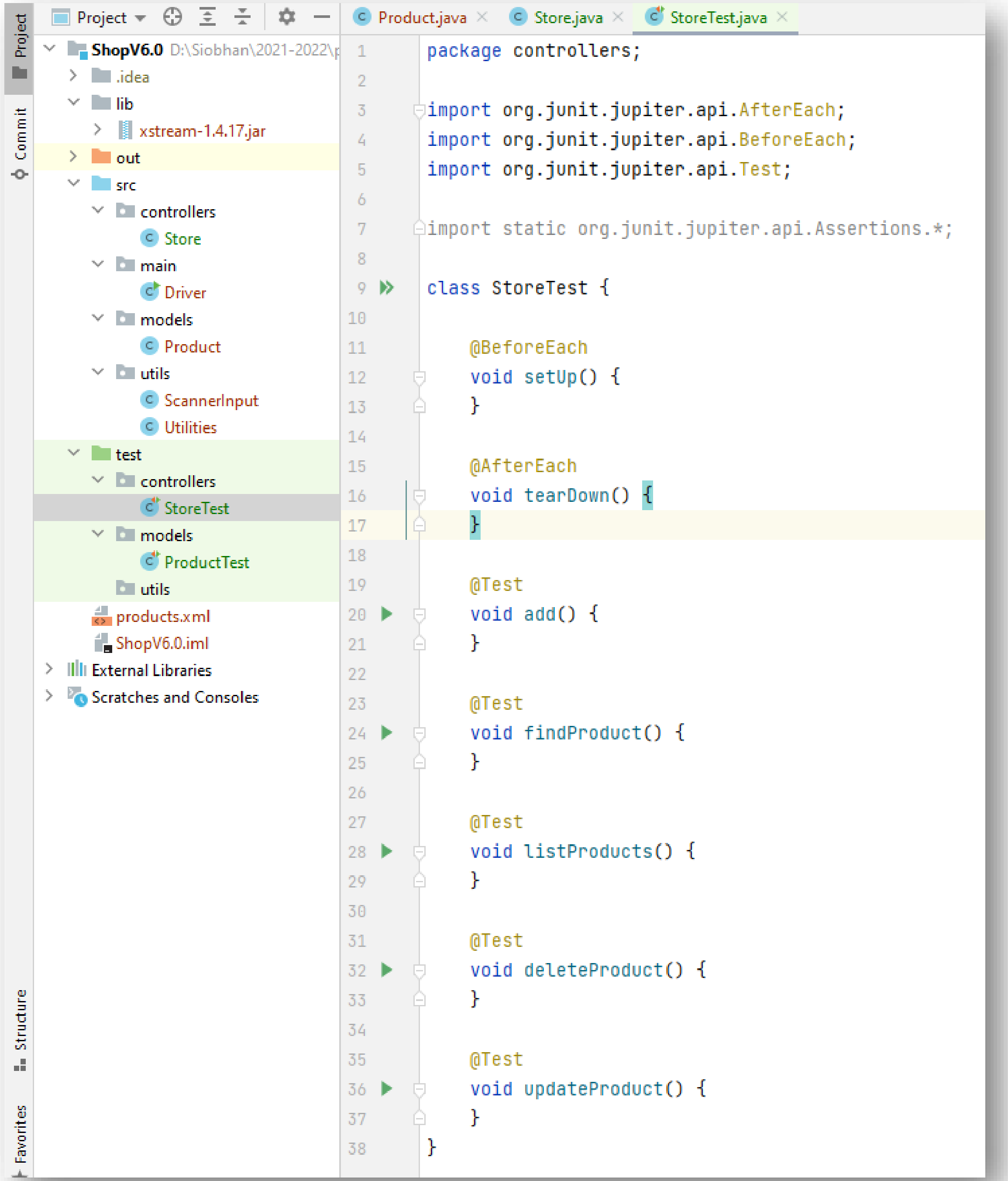
Open Store.java and call “Create Test”

Call the test class, **StoreTest**

Generate the default **setUp()** and **tearDown()** methods and also generate test methods for selected member methods.



Generated StoreTest.java




```java
package controllers;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class StoreTest {

    @BeforeEach
    void setUp() {
    }

    @AfterEach
    void tearDown() {
    }

    @Test
    void add() {
    }

    @Test
    void findProduct() {
    }

    @Test
    void listProducts() {
    }

    @Test
    void deleteProduct() {
    }

    @Test
    void updateProduct() {
    }
}
```

Store.java – testing add(Product)

- Store
  - Store()
  - getProducts(): ArrayList<Product>
  - numberOfProducts(): int
  - isValidIndex(int): boolean
  - add(Product): boolean
  - findProduct(int): Product
  - listProducts(): String
  - deleteProduct(int): Product
  - updateProduct(int, String, int, double, boolean): boolean
  - cheapestProduct(): Product
  - listCurrentProducts(): String
  - averageProductPrice(): double
  - listProductsAboveAPrice(double): String
  - load(): void
  - save(): void
  - products: ArrayList<Product>

```java
public boolean add (Product product){
   return products.add (product);
}
```

testing add(Product)

```java
class StoreTest {

    private Product productBelow, productExact, productAbove, productZero;
    private Store storeWithProducts = new Store();
    private Store storeEmpty = new Store();

    @BeforeEach
    void setUp() {
        //name, 19 chars, code 999, unitCost 1, inCurrentProductLine true.
        productBelow = new Product("Television 42Inches", 999, 1, true);
        //name, 20 chars, code 1000, unitCost 999, inCurrentProductLine true.
        productExact = new Product("Television 50 Inches", 1000, 999, true);
        //name, 21 chars, code 10000, unitCost 1000, inCurrentProductLine false.
        productAbove = new Product("Television 60 Inches.", 10000, 1000, false);
        //name, 0 chars, code 9999, unitCost 0, inCurrentProductLine false.
        productZero = new Product("", 9999, 0, false);

        storeWithProducts.add(productBelow);
        storeWithProducts.add(productExact);
        storeWithProducts.add(productAbove);
    }

    @AfterEach
    void tearDown() {
        productBelow = productExact = productAbove = productZero = null;
        storeEmpty = storeWithProducts = null;
    }
```

```java
@Test
void addingToAnArrayListThatHasProductsIsSuccessful() {
    assertEquals(3, storeWithProducts.numberOfProducts());
    assertTrue(storeWithProducts.add(productZero));
    assertEquals(4, storeWithProducts.numberOfProducts());
    assertEquals(productZero, storeWithProducts.findProduct(3));
}

@Test
void addingToAnArrayListThatHasNoProductsIsSuccessful() {
    assertEquals(0, storeEmpty.numberOfProducts());
    assertTrue(storeEmpty.add(productZero));
    assertEquals(1, storeEmpty.numberOfProducts());
    assertEquals(productZero, storeEmpty.findProduct(0));
}
```

Store.java – testing deleteProduct(Product)

Store
- Store()
- getProducts(): ArrayList<Product>
- numberOfProducts(): int
- isValidIndex(int): boolean
- add(Product): boolean
- findProduct(int): Product
- listProducts(): String
- deleteProduct(int): Product
- updateProduct(int, String, int, double, boolean): boolean
- cheapestProduct(): Product
- listCurrentProducts(): String
- averageProductPrice(): double
- listProductsAboveAPrice(double): String
- load(): void
- save(): void
- products: ArrayList<Product>

```java
public Product deleteProduct(int indexToDelete) {
   if (isValidIndex(indexToDelete)) {
      return products.remove(indexToDelete);
   }
   return null;
}
```

testing deleteProduct(Product)

```java
class StoreTest {

  private Product productBelow, productExact, productAbove, productZero;
  private Store storeWithProducts = new Store();
  private Store storeEmpty = new Store();

  @BeforeEach
  void setUp() {
    //name, 19 chars, code 999, unitCost 1, inCurrentProductLine true.
    productBelow = new Product("Television 42Inches", 999, 1, true);
    //name, 20 chars, code 1000, unitCost 999, inCurrentProductLine true.
    productExact = new Product("Television 50 Inches", 1000, 999, true);
    //name, 21 chars, code 10000, unitCost 1000, inCurrentProductLine false.
    productAbove = new Product("Television 60 Inches.", 10000, 1000, false);
    //name, 0 chars, code 9999, unitCost 0, inCurrentProductLine false.
    productZero = new Product("", 9999, 0, false);

    storeWithProducts.add(productBelow);
    storeWithProducts.add(productExact);
    storeWithProducts.add(productAbove);
  }

  @AfterEach
  void tearDown() {
    productBelow = productExact = productAbove = productZero = null;
    storeEmpty = storeWithProducts = null;
  }
```

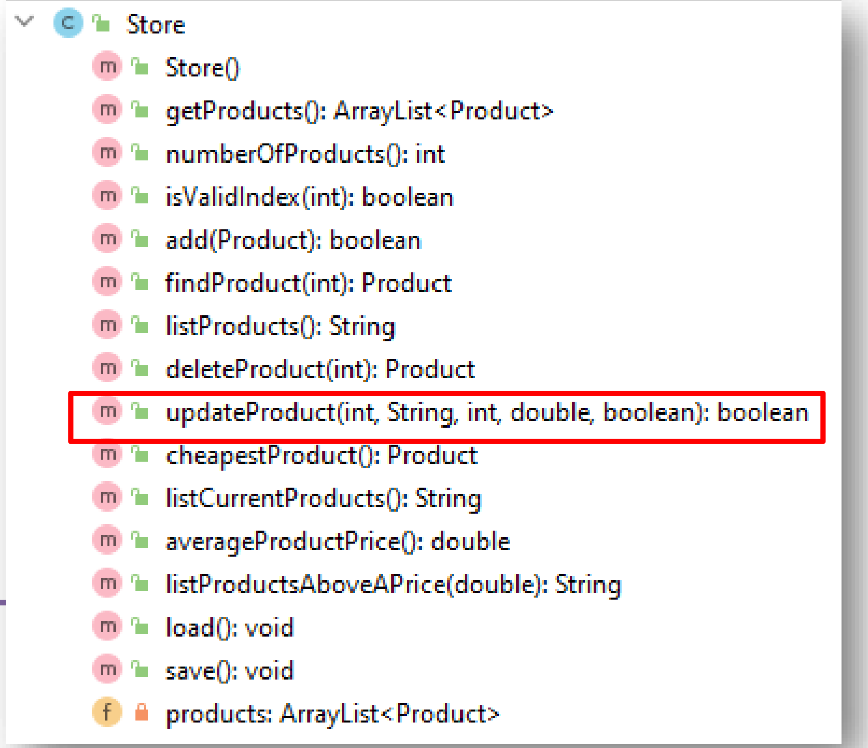
```java
@Test
void deletingAProductThatDoesNotExistReturnsNull(){
  assertNull(storeEmpty.deleteProduct(0));
  assertNull(storeWithProducts.deleteProduct(-1));
  assertNull(storeWithProducts.deleteProduct(3));
}

@Test
void deletingAProductThatExistsDeletesAndReturnsDeletedObject(){
  assertEquals(3, storeWithProducts.numberOfProducts());
  assertEquals(productAbove, storeWithProducts.deleteProduct(2));
  assertEquals(2, storeWithProducts.numberOfProducts());
}
```

Store.java – testing updateProduct

- Store
  - Store()
  - getProducts(): ArrayList<Product>
  - numberOfProducts(): int
  - isValidIndex(int): boolean
  - add(Product): boolean
  - findProduct(int): Product
  - listProducts(): String
  - deleteProduct(int): Product
  - updateProduct(int, String, int, double, boolean): boolean
  - cheapestProduct(): Product
  - listCurrentProducts(): String
  - averageProductPrice(): double
  - listProductsAboveAPrice(double): String
  - load(): void
  - save(): void
  - products: ArrayList<Product>

Store.java – testing updateProduct

```java
public boolean updateProduct(int indexToUpdate, String productName, int productCode, double unitCost, boolean inCurrentProductLine) {
   //find the product object by the index number
   Product foundProduct = findProduct(indexToUpdate);

   //if the product exists, use the details passed in the updateDetails parameter to
   //update the found product in the ArrayList.
   if (foundProduct != null) {
      foundProduct.setProductName(productName);
      foundProduct.setProductCode(productCode);
      foundProduct.setUnitCost(unitCost);
      foundProduct.setInCurrentProductLine(inCurrentProductLine);
      return true;
   }

   //if the product was not found, return false, indicating that the update was not successful
   return false;
}
```

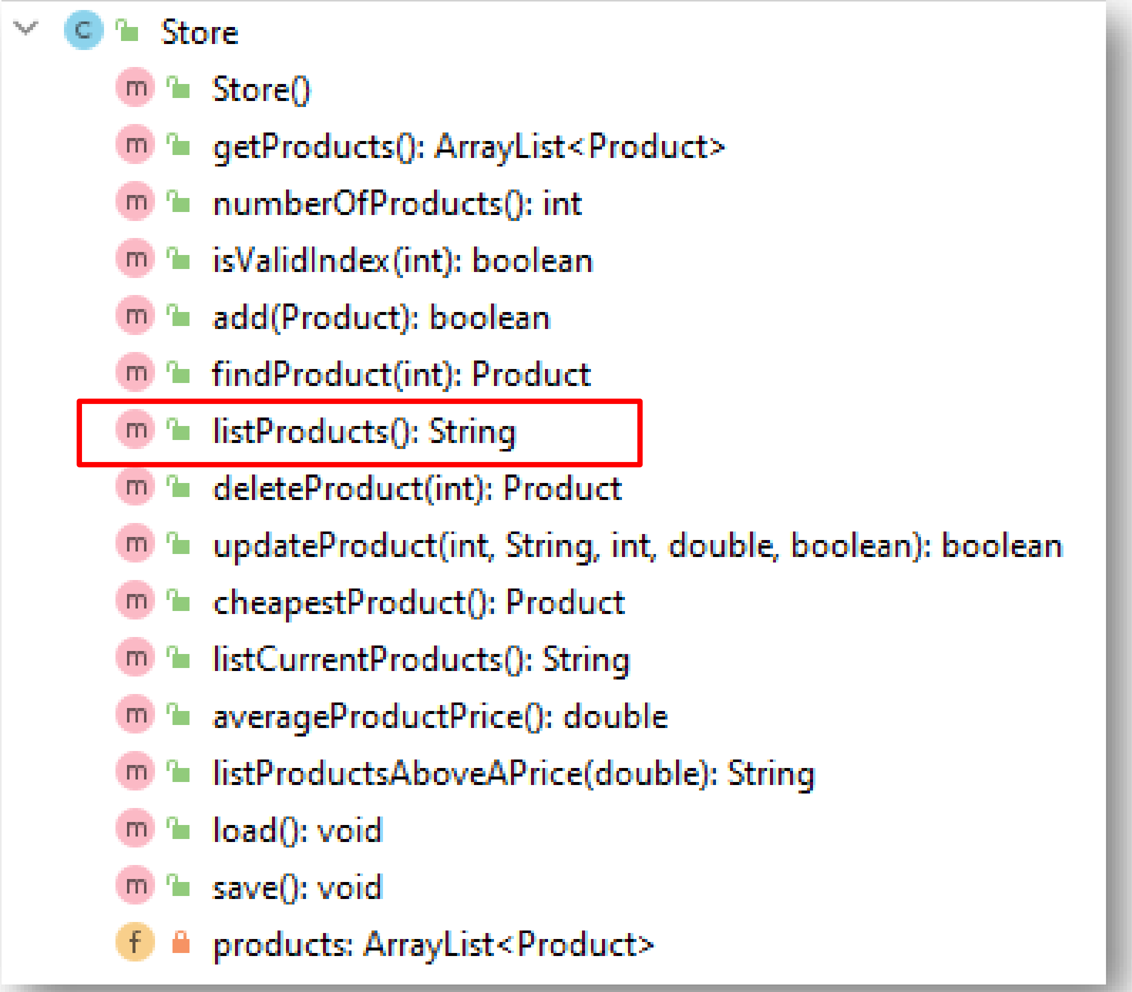


```java
class StoreTest {

    private Product productBelow, productExact, productAbove, productZero;
    private Store storeWithProducts = new Store();
    private Store storeEmpty = new Store();

    @BeforeEach
    void setUp() {
        //name, 19 chars, code 999, unitCost 1, inCurrentProductLine true.
        productBelow = new Product("Television 42Inches", 999, 1, true);
        //name, 20 chars, code 1000, unitCost 999, inCurrentProductLine true.
        productExact = new Product("Television 50 Inches", 1000, 999, true);
        //name, 21 chars, code 10000, unitCost 1000, inCurrentProductLine false.
        productAbove = new Product("Television 60 Inches.", 10000, 1000, false);
        //name, 0 chars, code 9999, unitCost 0, inCurrentProductLine false.
        productZero = new Product("", 9999, 0, false);

        storeWithProducts.add(productBelow);
        storeWithProducts.add(productExact);
        storeWithProducts.add(productAbove);
    }

    @AfterEach
    void tearDown() {
        productBelow = productExact = productAbove = productZero = null;
        storeEmpty = storeWithProducts = null;
    }
```

testing updateProduct

```java
@Test
void updatingANoteThatExistsReturnsTrueAndUpdates(){
    //check product index 2 exists and check the contents
    assertEquals(productAbove, storeWithProducts.findProduct(2));
    assertEquals("Television 60 Inches", storeWithProducts.findProduct(2).getProductName());
    assertEquals(-1, storeWithProducts.findProduct(2).getProductCode());
    assertEquals(1000, storeWithProducts.findProduct(2).getUnitCost());
    assertFalse(storeWithProducts.findProduct(2).isInCurrentProductLine());

    //update product 2 with new information and ensure contents updated successfully
    assertTrue(storeWithProducts.updateProduct(2, "Updating Product", 2000, 19.99, true));
    assertEquals("Updating Product", storeWithProducts.findProduct(2).getProductName());
    assertEquals(2000, storeWithProducts.findProduct(2).getProductCode());
    assertEquals(19.99, storeWithProducts.findProduct(2).getUnitCost());
    assertTrue(storeWithProducts.findProduct(2).isInCurrentProductLine());
}

@Test
void updatingAProductThatDoesNotExistReturnsFalse(){
    assertFalse(storeWithProducts.updateProduct(3, "Updating Product", 2, 19.99, false));
    assertFalse(storeWithProducts.updateProduct(-1, "Updating Product", 1002, 14.49, true));
    assertFalse(storeEmpty.updateProduct(0, "Updating Product", 1003, 199.99, false));
}
```

Store.java – testing listProduct()




```java
public String listProducts() {
   if (products.isEmpty()) {
      return "No products in the store";
   } else {
      String listOfProducts = "";
      for (int i = 0; i < products.size(); i++) {
         listOfProducts += i + ": " + products.get(i) + "\n";
      }
      return listOfProducts;
   }
}
```

testing listProduct()

```java
class StoreTest {

    private Product productBelow, productExact, productAbove, productZero;
    private Store storeWithProducts = new Store();
    private Store storeEmpty = new Store();

    @BeforeEach
    void setUp() {
        //name, 19 chars, code 999, unitCost 1, inCurrentProductLine true.
        productBelow = new Product("Television 42Inches", 999, 1, true);
        //name, 20 chars, code 1000, unitCost 999, inCurrentProductLine true.
        productExact = new Product("Television 50 Inches", 1000, 999, true);
        //name, 21 chars, code 10000, unitCost 1000, inCurrentProductLine false.
        productAbove = new Product("Television 60 Inches.", 10000, 1000, false);
        //name, 0 chars, code 9999, unitCost 0, inCurrentProductLine false.
        productZero = new Product("", 9999, 0, false);

        storeWithProducts.add(productBelow);
        storeWithProducts.add(productExact);
        storeWithProducts.add(productAbove);
    }

    @AfterEach
    void tearDown() {
        productBelow = productExact = productAbove = productZero = null;
        storeEmpty = storeWithProducts = null;
    }
```

```java
@Test
void listProductsReturnsNoProductsStoredWhenArrayListIsEmpty() {
    assertEquals(0, storeEmpty.numberOfProducts());
    assertTrue(storeEmpty.listProducts().toLowerCase().contains("no products"));
}

@Test
void listProductsReturnsProductsWhenArrayListHasProductsStored() {
    assertEquals(3, storeWithProducts.numberOfProducts());
    String productsString = storeWithProducts.listProducts();
    assertTrue(productsString.contains("Television 42Inches"));
    assertTrue(productsString.contains("Television 50 Inches"));
    assertTrue(productsString.contains("Television 60 Inches"));
}
```

Store.java – testing findProduct(int)

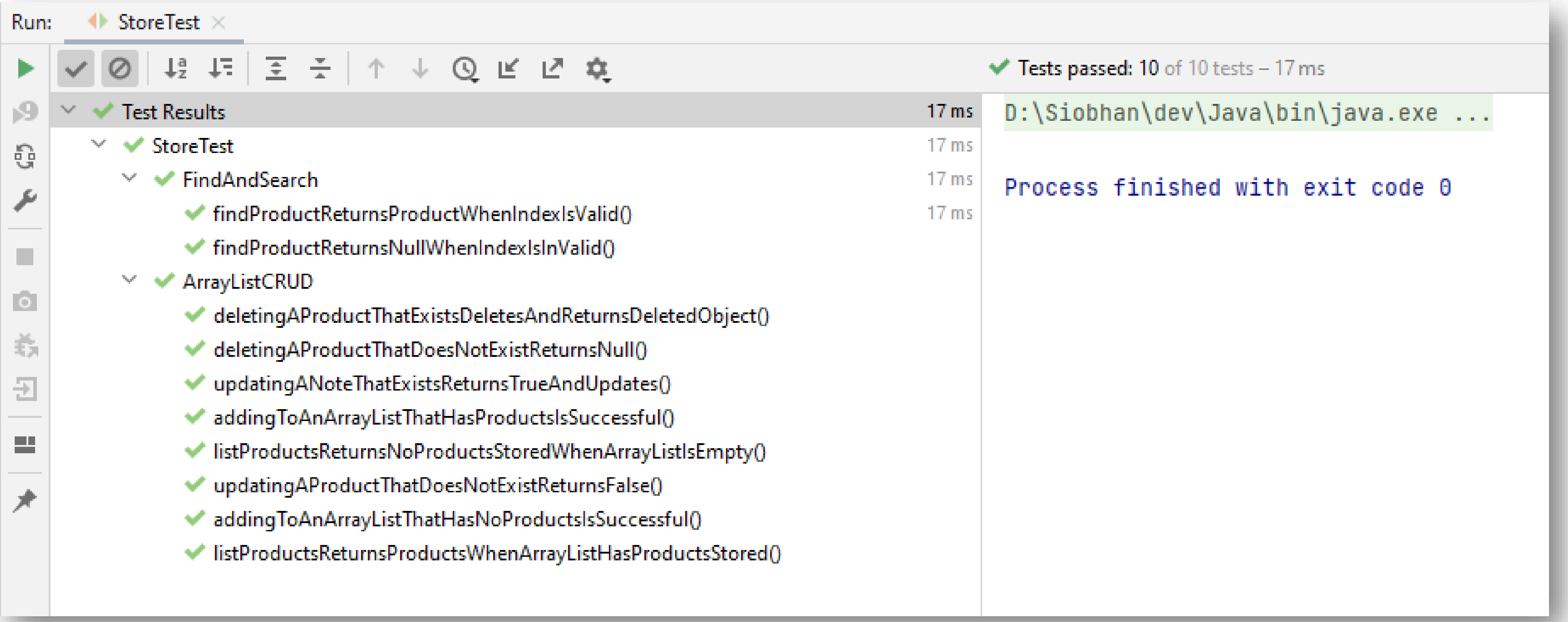
- Store
  - Store()
  - getProducts(): ArrayList<Product>
  - numberOfProducts(): int
  - isValidIndex(int): boolean
  - add(Product): boolean
  - findProduct(int): Product
  - listProducts(): String
  - deleteProduct(int): Product
  - updateProduct(int, String, int, double, boolean): boolean
  - cheapestProduct(): Product
  - listCurrentProducts(): String
  - averageProductPrice(): double
  - listProductsAboveAPrice(double): String
  - load(): void
  - save(): void
  - products: ArrayList<Product>

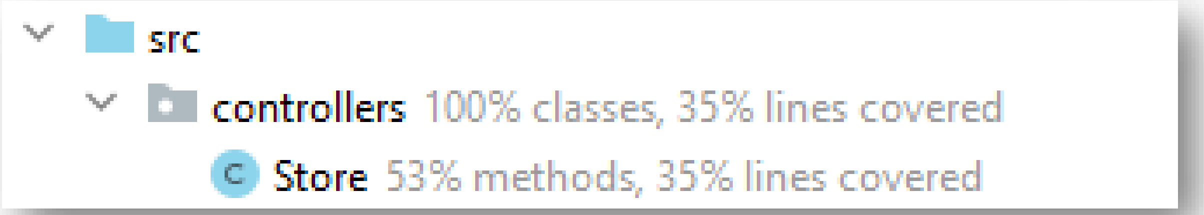
```java
public Product findProduct(int index) {
   if (isValidIndex(index)) {
      return products.get(index);
   }
   return null;
}
```

testing findProduct(int)

```java
class StoreTest {

    private Product productBelow, productExact, productAbove, productZero;
    private Store storeWithProducts = new Store();
    private Store storeEmpty = new Store();

    @BeforeEach
    void setUp() {
        //name, 19 chars, code 999, unitCost 1, inCurrentProductLine true.
        productBelow = new Product("Television 42Inches", 999, 1, true);
        //name, 20 chars, code 1000, unitCost 999, inCurrentProductLine true.
        productExact = new Product("Television 50 Inches", 1000, 999, true);
        //name, 21 chars, code 10000, unitCost 1000, inCurrentProductLine false.
        productAbove = new Product("Television 60 Inches.", 10000, 1000, false);
        //name, 0 chars, code 9999, unitCost 0, inCurrentProductLine false.
        productZero = new Product("", 9999, 0, false);

        storeWithProducts.add(productBelow);
        storeWithProducts.add(productExact);
        storeWithProducts.add(productAbove);
    }

    @AfterEach
    void tearDown() {
        productBelow = productExact = productAbove = productZero = null;
        storeEmpty = storeWithProducts = null;
    }
```

```java
@Test
void findProductReturnsProductWhenIndexIsValid() {
    assertEquals(3, storeWithProducts.numberOfProducts());
    assertEquals(productBelow, storeWithProducts.findProduct(0));
    assertEquals(productAbove, storeWithProducts.findProduct(2));
}

@Test
void findProductReturnsNullWhenIndexIsInValid() {
    assertEquals(0, storeEmpty.numberOfProducts());
    assertNull(storeEmpty.findProduct(0));
    assertEquals(3, storeWithProducts.numberOfProducts());
    assertNull(storeWithProducts.findProduct(-1));
    assertNull(storeWithProducts.findProduct(3));
}
```


Run: StoreTest
Tests passed: 10 of 10 tests – 17 ms
Test Results 17 ms
StoreTest 17 ms
FindAndSearch 17 ms
findProductReturnsProductWhenIndexIsValid() 17 ms
findProductReturnsNullWhenIndexIsInValid()
ArrayListCRUD
deletingAProductThatExistsDeletesAndReturnsDeletedObject()
deletingAProductThatDoesNotExistReturnsNull()
updatingANoteThatExistsReturnsTrueAndUpdates()
addingToAnArrayListThatHasProductsIsSuccessful()
listProductsReturnsNoProductsStoredWhenArrayListIsEmpty()
updatingAProductThatDoesNotExistReturnsFalse()
addingToAnArrayListThatHasNoProductsIsSuccessful()
listProductsReturnsProductsWhenArrayListHasProductsStored()
D:\Siobhan\dev\Java\bin\java.exe ...
Process finished with exit code 0


src
controllers 100% classes, 35% lines covered
Store 53% methods, 35% lines covered

Store.java

src
controllers 100% classes, 35% lines covered
Store 53% methods, 35% lines covered

```java
public class Store {

    private ArrayList<Product> products;

    public Store(){
        products = new ArrayList<Product>();
    }

    public ArrayList<Product> getProducts() {
        return products;
    }

    /**
     * This method returns the number of product objects stored in the ArrayList.
     *
     * @return An int value representing the number of product objects in the ArrayList.
     */
    public int numberOfProducts() { return products.size(); }

    /**
     * This method takes in a number and checks if it is a valid index in the products ArrayList.
     *
     * @param index A number representing a potential index in the ArrayList.
     * @return True of the index number passed is a valid index in the ArrayList, false otherwise.
     */
    public boolean isValidIndex(int index) {
        return (index >= 0) && (index < products.size());
    }

```

Topic List

- Product and ProductTest.java
- JUnit Testing of Store.java
- Testing Driver.java

Driver.java (and ScannerInput.java)

- JUnit is not used to test the class that takes input from the console.

- Why do you think this is?

Any Questions?